

Tema 1. Segmentación

Formatos de instrucción

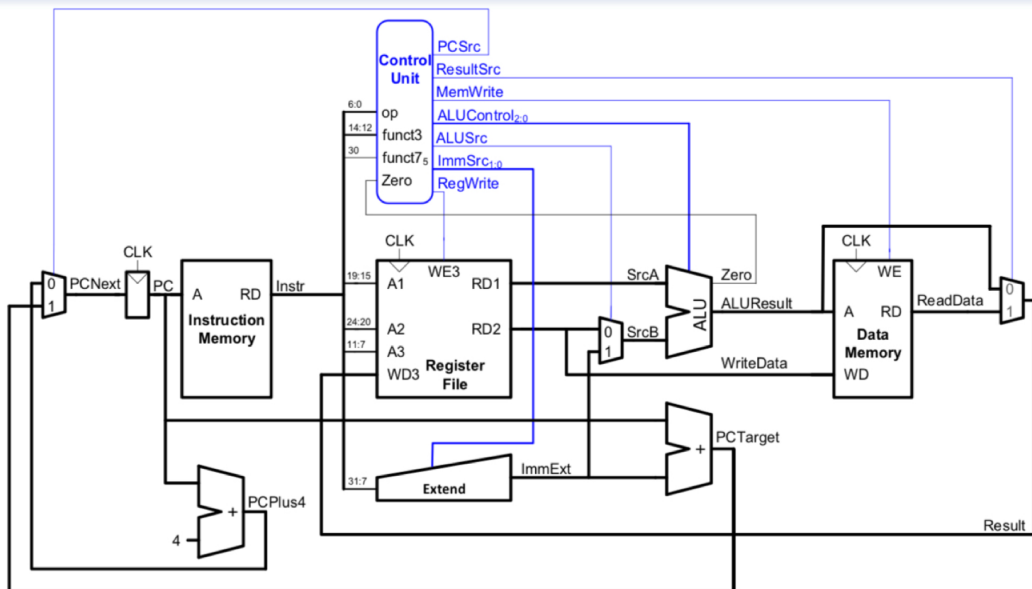
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
funct7	rs2	rs1	funct3	rd	op
imm _{11:0}		rs1	funct3	rd	op
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
imm _{31:12}				rd	op
imm _{20,10:1,11,19:12}				rd	op
20 bits		5 bits		7 bits	

R-Type
I-Type
S-Type
B-Type
U-Type
J-Type

R-Type: add, sub, sll, srl, sllw, sllr, sllr, sllr, or, and...
 I-Type: addi, slli, sli, sliw, sliw, sliw, sliw, ori, andi, lw...
 S-Type: sllw
 B-Type: bge, bne, blt, bge, bltu, bgeu
 U-Type: lui
 J-Type: jal



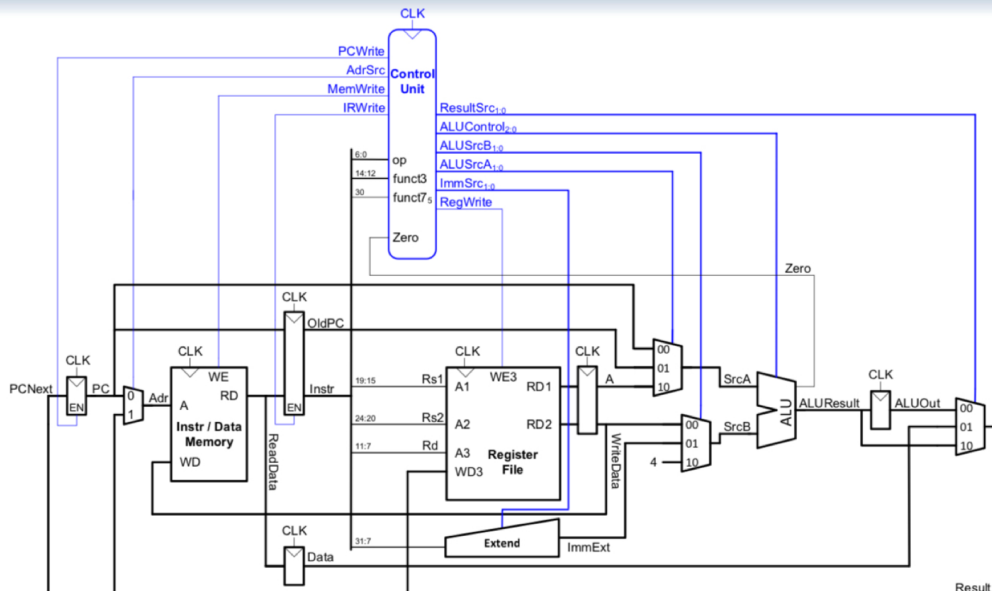
RUTA DE DATOS Y CONTROL MONOCICLO



8



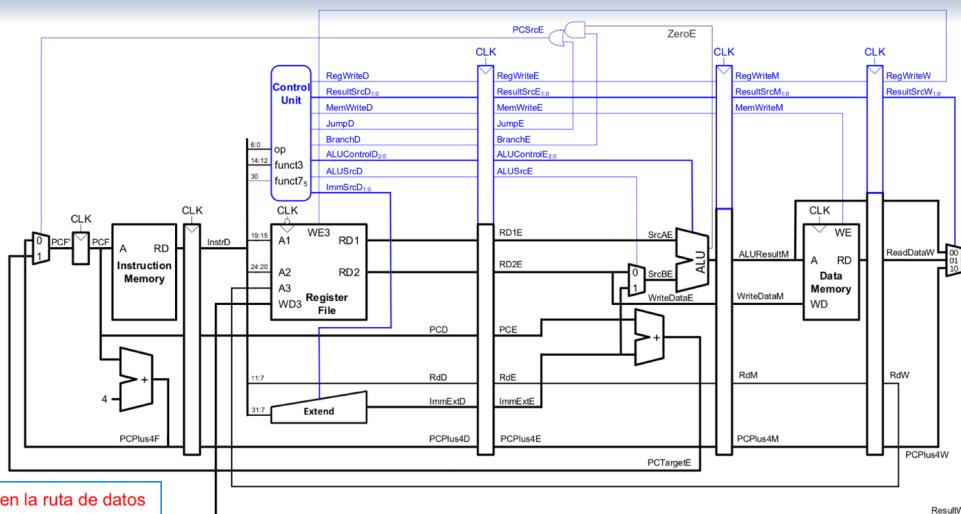
RUTA DE DATOS Y CONTROL MULTICICLO



10



UNIDAD DE CONTROL SEGMENTADA



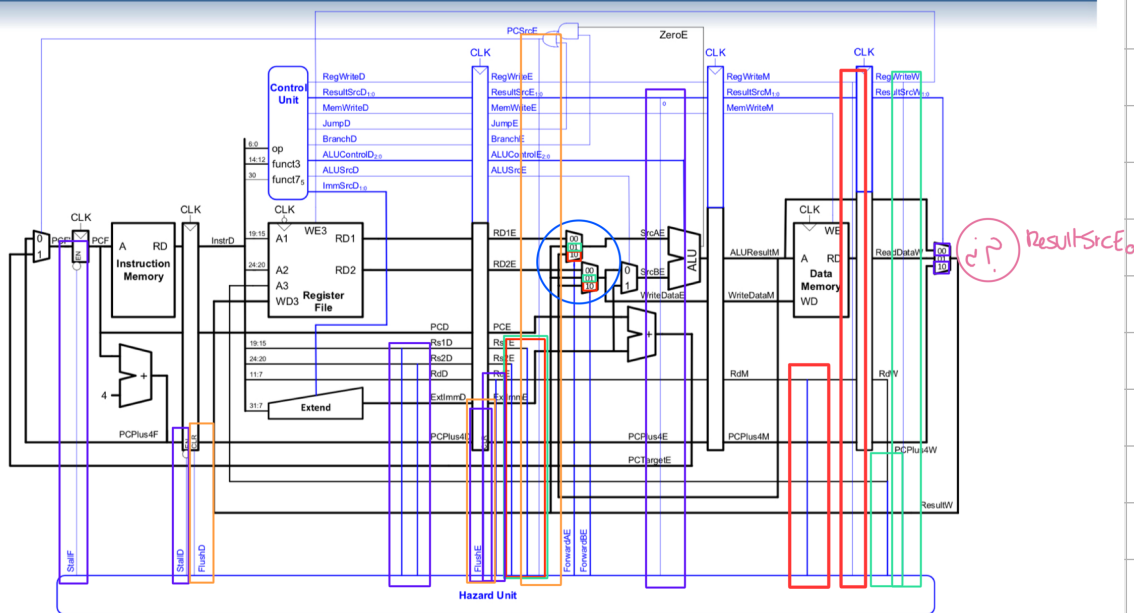
Ver que ocurre en la ruta de datos al ejecutar el siguiente código:

```
add x3, x9, x11
lw x5, 4(x7)
sub x5, x6, x8
ori x6, x7, 4
```

25



RISC-V: DETECCIÓN Y SOLUCIÓN DE RIESGOS



48

* $Rs1E \sim Rs2E$ * $ForwardAE(Rs1E) \sim ForwardBE(Cr2E)$

Cortocircuito (forwarding) realimentación

mismo registro escribe en registro

● if $((Rs1E == Rd1) \&\& RegWriteM) \&\& (Rs1E != 0) I_{i+1}$

ForwardAE=10

● else if $((Rs1E == Rd2) \&\& RegWriteW) \&\& (Rs1E != 0) I_{i+2}$

ForwardAE=01

else

ForwardAE=00 normal

● Parada (stall) detener IF < 10 y anular EX durante LW

$WnStall = ResultSrcE_0 \&\& ((Rs1D == RdE) \parallel (Rs2D == RdE))$

$stallF = stallD = flushE = WnStall$

detener

clear

¿por qué debe realizar esa compar.?

¿P Sabiendo el bit de ResultSrcE, ¿es seguro saber que se trata de una instrucción lw?

Riesgos de control

● Inhibición (flush) borrar las instrucciones siguientes al SALT0 si se realiza

$flushD = PCSrcE$

$flushE = WnStall \parallel PCSrcE$

Riesgo de datos

Riesgos de datos

- LDE lectura después de escritura
- EDL escritura después de lectura
- EDE escritura después de escritura

LDE

add x1, x2, x3 (escribe en registro x1)
add x4, x1, x2 (lee registro x1)
↳ no puede utilizar x1 hasta que se escriba en el registro

Se necesita que la escritura termine (wb)
hasta poder leer.
Se puede leer (ID) en el mismo ciclo de escritura

Soluciones

- Introducir instrucciones **Nop** add x0, x0, x0 entre las dependencias
- Reordenar el código para dejar las dependencias
- Detener el pipeline durante ID
↳ Inst. anteriores a etapa de parada se paran
↳ Inst. posteriores a etapa de parada siguen
IF₀, ID₀
- Cortocircuito (forwarding): realimentar datos desde etapa EX ya que ya están calculados en esa etapa

EDL

add x1, x4, x3 (lee registro x4)
add x4, x5, x2 (escribe registro x4)
↳ si escribe antes de que lo lea la 1ª instrucción

EDE

add x4, x2, x3 (escribe registro x4)
add x4, x1, x2 (escribe registro x4)
↳ si escritura de la 1ª instrucción ocurre después de la 2ª escritura

NO se produce cuando todas las instrucciones tienen igual duración

Operaciones multiciclo instrucciones tardan más de un ciclo de ejecución en Punto Flotante

Latencia de las UFs (Unidades Funcionales) nº ciclos entre instrucción que genera dato e instrucción que lo usa
Intervalo de iniciación nº ciclos que deben pasar entre dos ejecuciones de un mismo tipo de instrucción.
Depende si UF es segmentada o no

Problemas

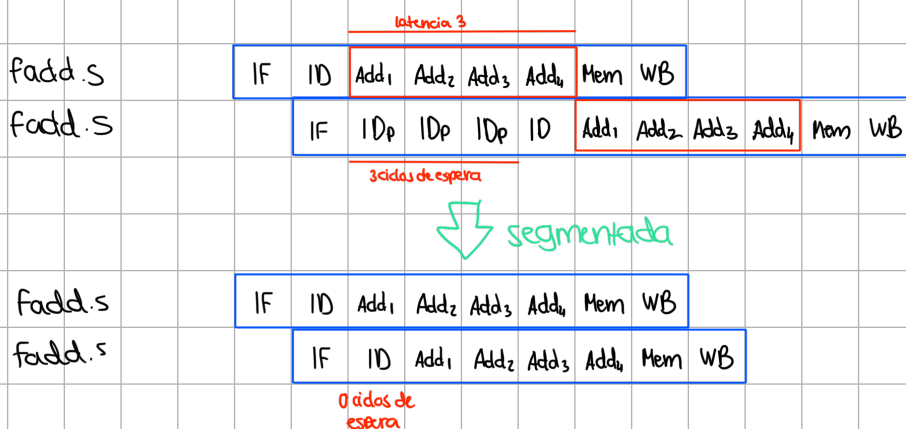
- Riesgos estructurales
- Mayor penalización de riesgos LDE
- Aparecen riesgos EDE y EDL
- Finalización fuera de orden

Soluciones

Riesgos estructurales > dos instrucciones necesitan la misma UF

- No segmentada: esperar que la UF haya acabado la primera operación

- Segmentar UFs con latencia > 1
↳ División: detectar riesgo y realizar paradas
- Esperar



- Dos instrucciones no pueden acceder a la vez a las etapas Mem y WB

fmul.s	IF	ID	Mul ₁	Mul ₂	Mul ₃	Mul ₄	Mem	WB	UF	Tiempo de ejecución
fadd.s		IF	ID	Add ₁	Add ₂	Add ₃	Mem	WB	FP add	
									FP mul	

3
4

⚡ Añadir parada

fmul.s			IF	ID	Mul ₁	Mul ₂	Mul ₃	Mul ₄	Mem	WB
fadd.s			IF	ID _p	ID	Add ₁	Add ₂	Add ₃	Mem	WB

finalización fuera de orden instrucciones pueden acabar en un orden diferente al de lanzamiento

- Escritura simultánea en el BR
- Riesgos escritura EDE

fadd.s F ₂ F0F8	IF	ID	Add ₁	Add ₂	Add ₃	Add ₄	Mem	WB					
fmul.s F0F4 F6		IF	ID	Mul ₁	Mul ₂	Mul ₃	Mul ₄	Mul ₅	Mul ₆	Mul ₇	Mem	WB	
flw F ₂ 0(x2)			IF	ID	EX	Mem	WB						

Solución 1: añadir paradas

fadd.s F ₂ F0F8	IF	ID	Add ₁	Add ₂	Add ₃	Add ₄	Mem	WB						
fmul.s F0F4 F6		IF	ID	Mul ₁	Mul ₂	Mul ₃	Mul ₄	Mul ₅	Mul ₆	Mul ₇	Mem	WB		
flw F ₂ 0(x2)			IF	ID ₀	ID _p	ID	EX	Mem	WB					

2 ciclos de espera

Solución 2: inhibir escritura de la 1ª instrucción

fadd.s F ₂ F0F8	IF	ID	Add ₁	Add ₂	Add ₃	Add ₄	Mem	WB				
fmul.s F0F4 F6		IF	ID	Mul ₁	Mul ₂	Mul ₃	Mul ₄	Mul ₅	Mul ₆	Mul ₇	Mem	WB
flw F ₂ 0(x2)			IF	ID	EX	Mem	WB					

Riesgos EDL

- NO se producen: lectura siempre se realiza en ID y escritura siempre se realiza en WB

Rendimiento de los procesadores

Medida del rendimiento

Métrica: tiempo de ejecución (tiempo que tarda en realizar una tarea) y productividad (tareas realizadas por unidad de tiempo)

patrón de medida: establece programas para realizar la medida (benchmarks)

→ n° instrucciones totales

Tiempo de ejecución $T = N * CPI * T_c$ → tiempo de ciclo
↳ n° medio de ciclos por instrucción

$$CPI_{real} = CPI_{ideal} + media \sum \text{penalizaciones por instrucción}$$

$$Speed-up = \frac{n^{\circ} \text{instrucciones} * n^{\circ} \text{etapa}}{n^{\circ} \text{instrucciones} * CPI}$$

$$Eficiencia = \frac{Speed-up_{real}}{Speed-up_{ideal}}$$

Medidas del rendimiento: MIPS, MFLOPS